
Redes Neurais e Aprendizagem Profunda

— Rede Neuronal Artificial —

Sumário

Neurónio biológico

Computações lógicas

Perceptrão

Perceptrão multicamada e
backpropagation

MLPs para regressão

MLPs para classificação

Rede Neuronal Artificial

- modelo de AA inspirado na rede de neurónios biológicos do cérebro
 - gradualmente tornou-se muito diferente...
- Essência da Aprendizagem Profunda
- Características
 - versátil
 - poderoso
 - escalável
- Ideal para problemas grandes e altamente complexos
 - classificação de milhões de imagens
 - serviços de reconhecimento de voz
 - recomendação dos melhores vídeos
 - vencer o campeão mundial de Go

História

- Warren McCulloch e Walter Pitts, 1943
 - *A Logical Calculus of Ideas Immanent in Nervous Activity*
 - Modelo computacional para cálculos complexos, usando lógica proposicional
- Evolução
 - 1950: crença de criação de máquinas inteligentes
 - 1960: 1º inverno
 - 1980: proposta de novas arquiteturas
 - 1990: 2º inverno
 - 2010: nova onda de interesse

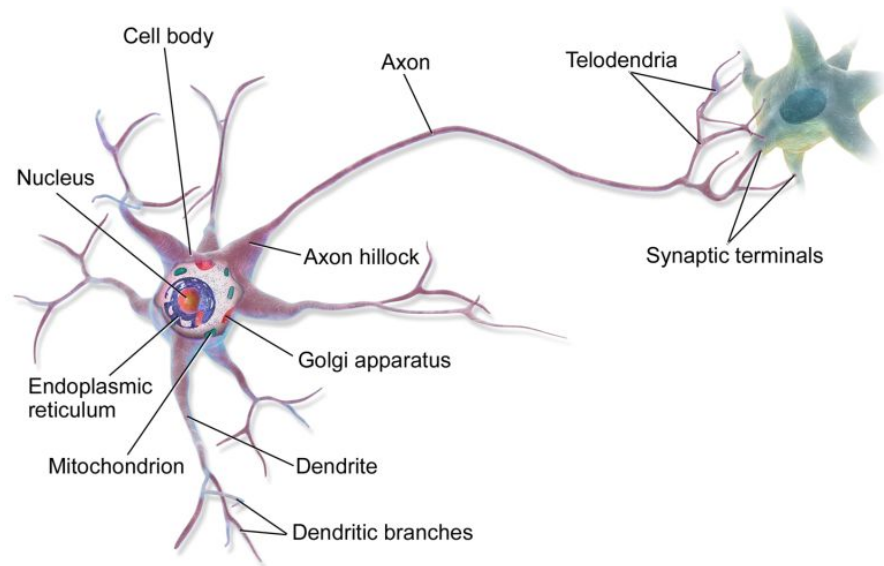
Agora será diferente?

- Enorme quantidade de dados
 - RNA superam outros modelos em problemas muito grandes e complexos
- Tremendo poder computacional
 - *Moore's law*: n° componentes nos circuitos integrados (duplicação de componentes na mesma área de semi-condutor em cada 18 meses)
 - Indústria dos jogos: produção de placas GPU
 - Plataformas cloud: poder acessível a todos
- Melhores algoritmos de treino
- Limitações teóricas mostraram ser benignas
 - prisão em mínimos locais
- Ciclo de financiamento e progresso

Neurónio Biológico

Neurónio biológico

- Corpo da célula
 - núcleo e restantes componentes complexos
- Dendrites
 - extensões ramificadas do núcleo
- Axónio
 - extensão longa
 - comprimento
 - desde algumas vezes maior que o corpo
 - até dezenas de milhar
- Telodêndria
 - ramificações do axónio
- Terminais sinápticos (sinapses)
 - estruturas terminais
 - "ligadas" a dendrites de outras células



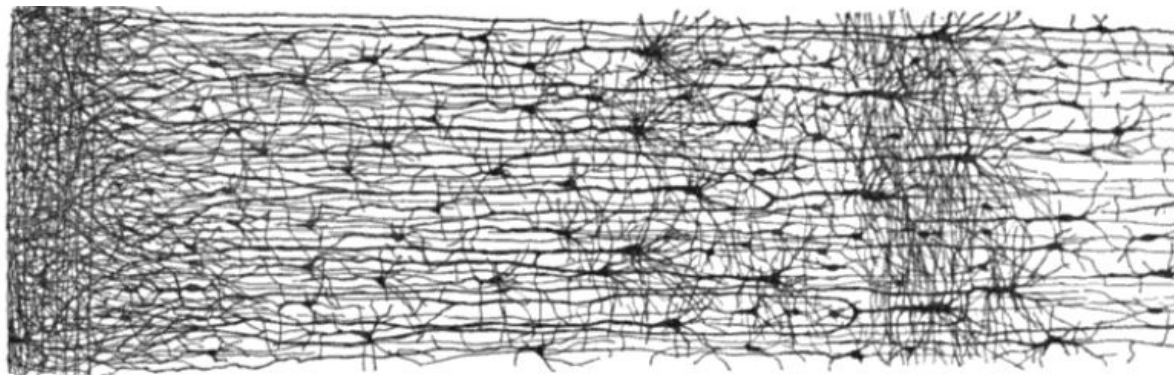
fonte:
<https://en.wikipedia.org/wiki/Neuron>

Funcionamento

- impulsos elétricos curtos viajam pelos axónios
 - potenciais de ação
- sinapses libertam sinais químicos
 - neurotransmissores
- neurónios disparam os impulsos
 - quando recebem quantidade suficiente de neurotransmissores em pouco milisegundos

Organização

- Estrutura
 - rede de milhares de milhões de neurónios
 - cada neurónio ligado a milhares de outros neurónios
- Arquitetura
 - área ativa de investigação
 - indícios de estarem organizados em camadas consecutivas (cortex cerebral)

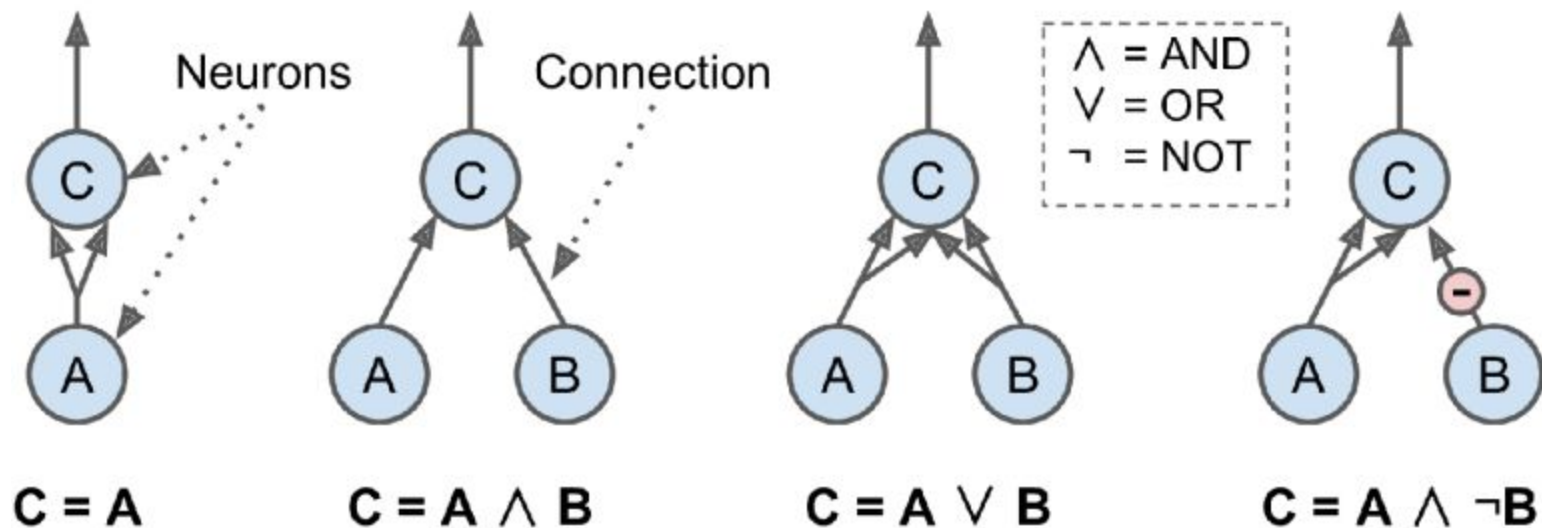


Computações lógicas

Neurónio artificial

- Estrutura
 - uma ou mais entradas binárias
 - uma saída binária
- Ativação
 - saída ativa quando mais de um determinado nº de entradas estão ativas
- Rede de neurónios capaz de realizar qualquer função lógica

Exemplos



assumir que neurónio é ativado quando pelo menos duas entradas estão ativas

Percepção

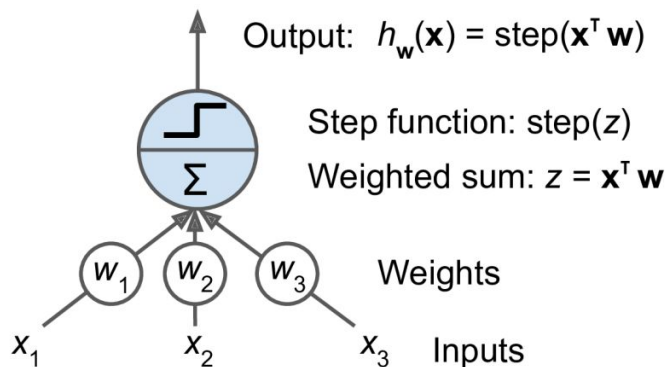
Perceptrão

- Uma das arquiteturas mais simples
 - Frank Rosenblatt, 1957
- Algo diferente do neurónio artificial
- Baseado na unidade lógica de limite
 - TLU: threshold logic unit
 - LTU: linear threshold unit

TLU

Estrutura

- entradas e saídas são reais
- cada ligação tem um peso associado
- calcula a soma pesada das entradas
- aplica uma função de degrau
- emite o resultado



Função

- heaviside
- sinal

$$\text{heaviside}(z) = \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases}$$

$$\text{sgn}(z) = \begin{cases} -1 & \text{if } z < 0 \\ 0 & \text{if } z = 0 \\ +1 & \text{if } z > 0 \end{cases}$$

TLU e classificação

- Modelo linear binário
 - calcula a combinação linear das entradas
 - resultado maior que limite $\rightarrow$ classe positiva
 - caso contrário $\rightarrow$ classe negativa
- Treino
 - encontrar os valores corretos para $w_0, w_1, \dots, w_n$

Perceptrão

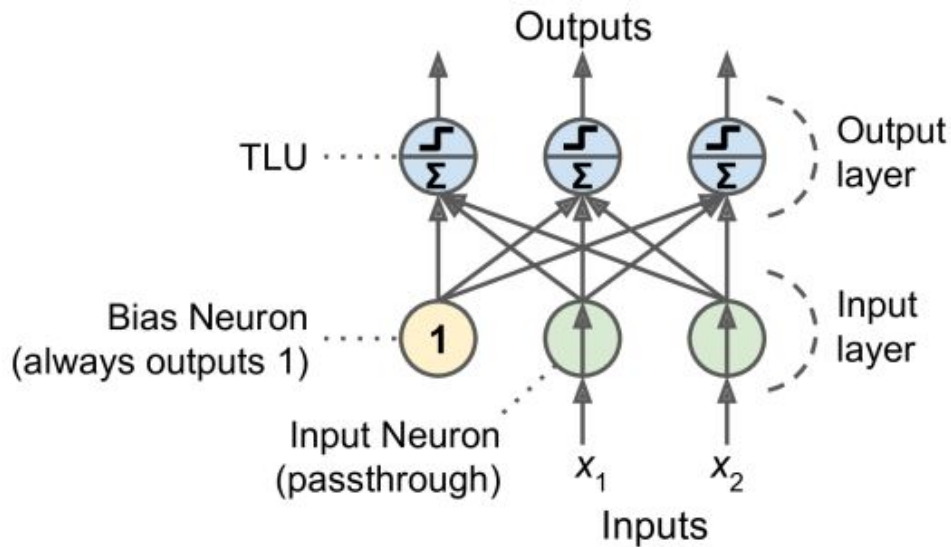
- Arquitetura
 - Uma camada de TLUs
 - cada TLU ligado a todas as entradas
- Camada densa
 - todos os neurónios ligados a todos os neurónios da camada anterior
 - dense layer, fully connected layer

Perceptrão = camada densa de TLUs

- Camada de entrada
 - neurónios *passthrough*
 - emite o que recebe à entrada
 - neurónio *bias*
 - ligado a atributo bias (x_0)
 - emite sempre 1

Exemplo

- Arquitetura
 - 2 entradas
 - 3 saídas
- Classificador multi-saída
 - permite classificar instâncias em 3 classes binárias simultaneamente



Valores de saída

- Saídas de uma camada
 - Cálculo simultâneo para várias instâncias
- Cálculo de matrizes

$$h_{\mathbf{W},\mathbf{b}}(\mathbf{X}) = \phi(\mathbf{X}\mathbf{W} + \mathbf{b})$$

- **X**: matriz
 - atributos de entrada
 - uma linha por instância
 - uma coluna por atributo
- **W**: matriz
 - todos os pesos, exceto do neurónio bias
 - uma linha por neurónio de entrada
 - uma coluna por neurónio artificial na camada
- **b**: vetor
 - todos os pesos entre o neurónio bias e os neurónios artificiais
- **Φ** : função
 - função de ativação
 - TLU: função signal

Treino do perceptrão

- Algoritmo proposto por Rosenblatt
 - inspirado na regra de Hebb
- Regra de Hebb
 - "cells that fire together, wire together"
 - quando um neurónio ativa outro neurónio várias vezes, a ligação entre ambos torna-se mais forte
 - o peso de ligação entre 2 neurónios tende a aumentar quando disparam simultaneamente
- Regra do perceptrão
 - tem em consideração o erro da rede quando faz uma previsão
 - reforça ligações que ajudam a reduzir o erro

Regra de treino

Algoritmo

- Para cada instância de treino é feita a previsão
- Para cada neurónio de saída que produz um erro
 - reforça os pesos das ligações de entrada (atributos) que contribuíram para a previsão correta

$$w_{i,j}^{(\text{next step})} = w_{i,j} + \eta(y_j - \hat{y}_j)x_i$$

- $w_{i,j}$
 - peso entre o neurónio de entrada i e o neurónio de saída j
- x_i
 - valor de entrada i da instância corrente
- $\hat{y}_j$
 - valor do neurónio de saída j da instância corrente
- y_j
 - valor objetivo do neurónio de saída j da instância corrente
- η
 - learning rate

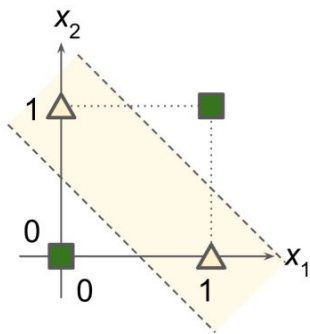
Algoritmo de aprendizagem

- Muito semelhante à descida do gradiente estocástica
- scikit-learn
 - `Perceptron()` =
`SGDClassifier(loss="perceptron", learning_rate="constant", eta0=1, penalty=None)`
- Perceptrão não emite probabilidades (ao contrário da regressão logística)
 - faz previsões com base num limite concreto

Limitação e solução

Limitação

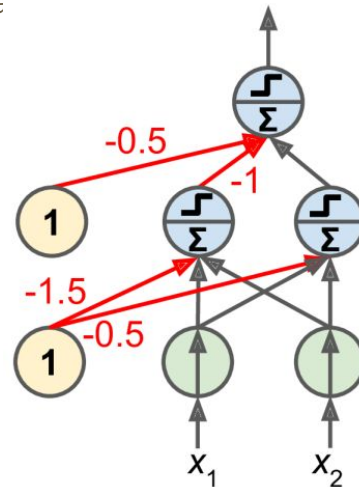
- incapaz de resolver problemas triviais
 - XOR



- ... como qualquer outro modelo de classificação linear!

Solução

- empilhar vários perceptrão
- XOR
 - todas as ligações têm peso 1 exceto as indica ' '



Perceptrão multi-camada e backpropagation

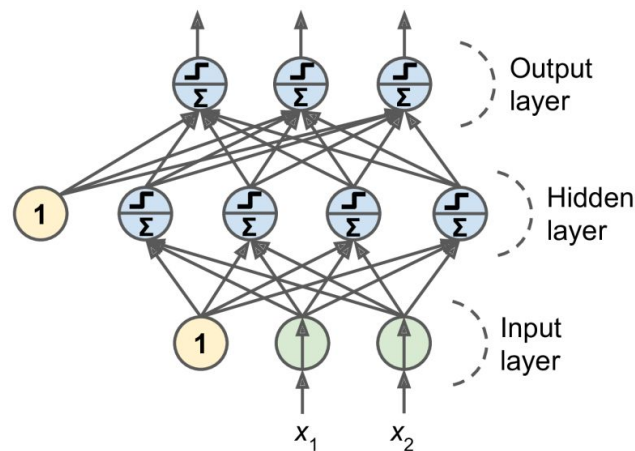
Perceptrão multi-camada

Estrutura

- camada de entrada
 - uma camada passthrough
- camadas escondidas
 - uma ou mais camadas de TLUs
- camada de saída
 - uma camada final de TLUs
- todas as camadas (exceto a de saída)
 - têm um neurónio bias
 - são densas

Convenção

- lower layers
 - camadas mais próximas da entrada
- upper layers
 - camadas mais próximas da saída



Treino de MLP

- Algoritmo **backpropagation**
 - David Rumelhart, Geoffrey Hinton, and Ronald Williams, 1986. *Learning Internal Representations by Error Propagation*.
 - Descida do gradiente com **técnica eficiente para cálculo automático** dos gradientes
- Descida do gradiente
 - calcula o gradiente do erro
 - atualiza os pesos de acordo com o gradiente
- Técnica
 - reverse-mode autodiff (uma das técnicas para cálculo automático dos gradientes)
 - calcula a alteração necessária para cada peso e bias da rede (que permite reduzir erro) em **2 passos sobre a rede**: forward, backward

Backpropagation

- Algoritmo
 - para cada instância
 - passagem *forward*: para cada instância faz uma previsão e calcula o erro
 - passagem *backward*: volta a cada camada para trás medindo a contribuição de cada ligação para o erro (gradiente do erro)
 - passo descida do gradiente: atualiza os pesos das ligações para reduzir o erro
- Pesquisa dos pesos
 - considera um mini-batch de cada vez
 - "percorre" todo o conjunto de treino várias vezes
 - cada passagem é chamada **época**

Passo do algoritmo

1. Passagem forward
 - a. cada mini-batch passa pela camada de entrada e é enviado para a primeira camada escondida
 - b. para cada instância, calcula a saída de todos os neurónios da camada escondida e o resultado é passado à camada seguinte até chegar à camada de saída
 - i. equivalente a fazer uma previsão mas todos os resultados intermédios são mantidos para serem usados na passagem backwards
2. Cálculo do erro da saída
 - a. utiliza função de perda para comparar a saída atual com a desejada e devolve uma medida de erro
3. Passagem backward
 - a. calcula, de forma analítica, a contribuição de cada saída para o erro (chain rule)
 - b. mede quanto destas contribuições provém de cada ligação da camada anterior (usando chain rule) até chegar à camada de entrada, propagando o gradiente do erro pela rede para trás
4. Descida do gradiente
 - a. atualiza todos os pesos das ligações da rede usando os gradientes do erro calculados

Função de ativação

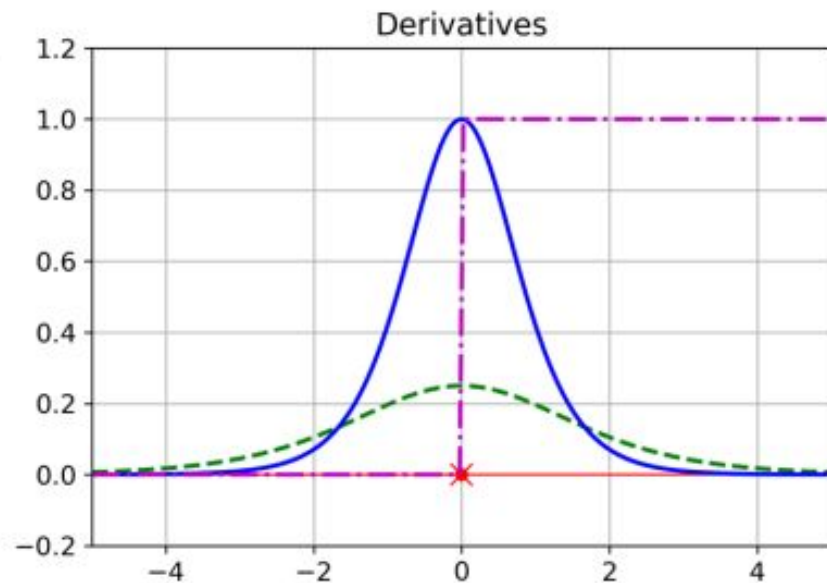
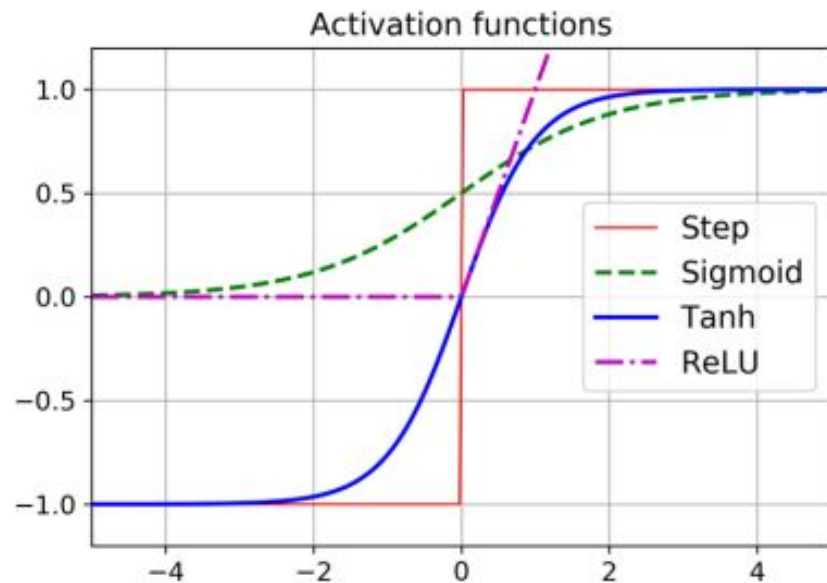
- Função step
 - contém apenas segmentos planos, não existindo gradiente...
- Função logística (sigmoide)
 - substituição original
 - derivada bem definida e diferente de zero em todos os pontos
 - permite a descida do gradiente
- Funciona com outras funções
 - tangente hiperbólica, $\tanh()$
 - unidade linear retificada, $\text{ReLU}()$
 - embora não tenha derivada para $z=0$ é muito rápida de calcular
 - ajuda a resolver alguns problemas da descida do gradiente
 - tornou-se a função por omissão

$$\sigma(z) = 1 / (1 + \exp(-z)).$$

$$\tanh(z) = 2\sigma(2z) - 1$$

$$\text{ReLU}(z) = \max(0, z)$$

Funções de ativação e suas derivadas



Observações

- Função de ativação
 - elemento **essencial** da RNA
 - senão não passa de uma combinação linear das entradas
- Inicialização dos pesos
 - é essencial a **inicialização aleatória** dos pesos
 - se forem zero
 - todos os neurónios de uma camada são perfeitamente iguais
 - o backpropagation afeta todos de igual forma
 - permanecem idênticos
- Tipo de arquitetura
 - feed-forward
 - o sinal flui apenas numa direção, das entradas para as saídas
- Rede profunda
 - 1990s
 - rede neuronal com mais de 2 camadas escondidas
 - atualmente
 - termo fuzzy
 - existem arquiteturas com dezenas e até centenas de camadas

MLPs para regressão

MLPs para regressão

- Prever um único valor
 - um neurónio de saída
 - a saída do neurónio é o valor previsto
- Regressão multi-variada
 - um neurónio de saída para cada dimensão de saída
 - Exemplos
 - localizar o centro de um objeto numa imagem: 2 saídas (posições x e y)
 - definir a bounding box de um objeto: 4 saídas: (x, y, largura, comprimento)

Funções

Função de ativação dos neurónios de saída

- gerar qualquer intervalo de valores
 - sem função de ativação
- restringir a valores positivos
 - ReLU
- intervalo específico de valores
 - função logística (ou tanh)
 - escalar as etiquetas para o intervalo apropriado

Função de perda

- Tipicamente MSE
 - mean squared error
 - erro quadrado médio
- Muitos outliers: MAE
 - mean absolute error
 - erro absoluto médio
- Alternativa: Huber loss
 - combinação de MSE e MAE
 - parâmetro: δ
 - $< \delta$: MSE (quadrática)
 - converge mais depressa
 - $> \delta$: MAE (linear)
 - menos sensível a outliers

Arquitetura típica para problemas de regressão

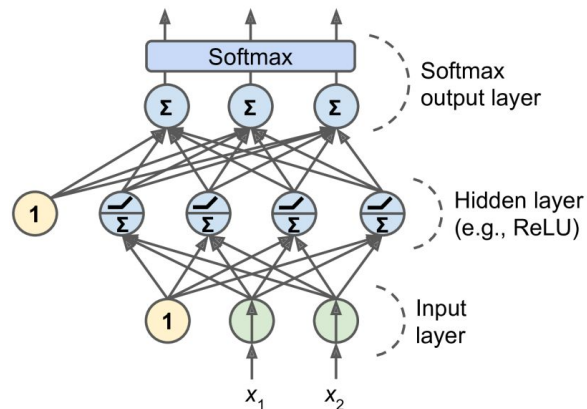
Hyperparameter	Typical Value
# input neurons	One per input feature (e.g., $28 \times 28 = 784$ for MNIST)
# hidden layers	Depends on the problem. Typically 1 to 5.
# neurons per hidden layer	Depends on the problem. Typically 10 to 100.
# output neurons	1 per prediction dimension
Hidden activation	ReLU (or SELU, see Chapter 11)
Output activation	None or ReLU/Softplus (if positive outputs) or Logistic/Tanh (if bounded outputs)
Loss function	MSE or MAE/Huber (if outliers)

MLP's para classificação

MLPs para classificação

- binária
 - um neurónio de saída com função de ativação **logística**
 - saída no intervalo $[0,1]$, pode ser interpretado como probabilidade estimada da classe positiva
- multi-etiqueta
 - um neurónio de saída para cada classe positiva com função de ativação logística
 - Exemplo
 - classificar mails como spam/not spam, urgente/não urgente

- multi-classe
 - um neurónio de saída para cada classe com função de ativação **softmax para toda a camada de saída**
 - assegura probabilidades estimadas no intervalo $[0,1]$ e que somam 1
 - Exemplo
 - classificação de imagens de algarismos



Funções

Função de ativação

- Função logística

$$\sigma(t) = \frac{1}{1 + \exp(-t)}$$

- Função softmax

- $\mathbf{x}$: exemplo
- k : classe
- $\theta^{(k)}$: pesos da classe k
- $\mathbf{s}(\mathbf{x})$: vetor dos scores para cada classe

$$s_k(\mathbf{x}) = \mathbf{x}^T \boldsymbol{\theta}^{(k)}$$

$$\hat{p}_k = \sigma(\mathbf{s}(\mathbf{x}))_k = \frac{\exp(s_k(\mathbf{x}))}{\sum_{j=1}^K \exp(s_j(\mathbf{x}))}$$

Função de perda

- Tipicamente cross-entropy
 - também designada *log loss* (duas classes)

$$J(\boldsymbol{\Theta}) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log(\hat{p}_k^{(i)})$$

Arquitetura típica para problemas de classificação

Hyperparameter	Binary classification	Multilabel binary classification	Multiclass classification
Input and hidden layers	Same as regression	Same as regression	Same as regression
# output neurons	1	1 per label	1 per class
Output layer activation	Logistic	Logistic	Softmax
Loss function	Cross-Entropy	Cross-Entropy	Cross-Entropy

Outras arquiteturas

Outras arquiteturas populares

- Hopfield Networks
- Boltzmann Machines
- Restricted Boltzmann Machines
- Deep Belief Nets
- Self-Organizing Maps

Bibliografia

Capítulo 10

*Hands-On Machine Learning with
Scikit-Learn, Keras & TensorFlow*
